pandas Data Cleaning

Every common fix, ready to copy. Keep it next to you while you work.

DATAMOUS
Cheat Sheet
Every line tested on the latest pandas

A GOOD ORDER TO CLEAN IN 1 Inspect > 2 Fix types > 3 Text > 4 Blanks > 5 Duplicates > 6 Outliers > 7 Check > 8 Save

LOAD & INSPECT

Read CSV / Excel	`df = pd.read_csv("data.csv")` `df = pd.read_excel("data.xlsx", sheet_name="Jan")`
Keep IDs as text	`df = pd.read_csv("data.csv", dtype={"zip": "str"})`
Read some columns	`df = pd.read_csv("data.csv", usecols=["id", "email"])`
Rows, types, blanks	`df.info()`
Column names	`df.columns.tolist()`
First 10 rows	`df.head(10)`
Stats per column	`df.describe(include="all")`
Count each value	`df["city"].value_counts(dropna=False)`
Unique per column	`df.nunique()`

WATCH pandas now reads text as the `str` dtype, not `object`.

MISSING VALUES

Blanks per column	`df.isna().sum()`
Share of blanks	`df.isna().mean().round(2)`
Rows with a blank	`df[df.isna().any(axis=1)]`
Mark fake blanks	`df = df.replace(["N/A", "-", ""], pd.NA)`
Drop if key is blank	`df = df.dropna(subset=["email"])`
Drop empty columns	`df = df.dropna(axis=1, how="all")`
Fill with a value	`df["city"] = df["city"].fillna("Other")`
Fill with median	`df["qty"] = df["qty"].fillna(df["qty"].median())`
Fill down	`df["region"] = df["region"].ffill()`
Fill by group	`df["qty"] = df["qty"].fillna(df.groupby("city")["qty"].transform("median"))`

WATCH `fillna(method="ffill")` is gone: use `.ffill()` or `.bfill()`.

DUPLICATES

Count repeats	`df.duplicated().sum()`
Repeats on a key	`df.duplicated(subset="email").sum()`
See every copy	`df[df.duplicated(keep=False)]`
Drop repeats	`df = df.drop_duplicates()` `df = df.drop_duplicates(subset="email")`
Keep latest per key	`df = df.sort_values("date")` `df = df.drop_duplicates("email", keep="last")`

WATCH Clean the text first, or `" Ann@x.com"` and `"ann@x.com"` stay apart.

TEXT CLEAN-UP

Trim extra spaces	`df["name"] = df["name"].str.strip()` `df["name"] = df["name"].str.replace(r"\s+", " ", regex=True)`
Trim all text cols	`cols = df.select_dtypes("str").columns` `df[cols] = df[cols].apply(lambda s: s.str.strip())`
lower / Title Case	`df["email"] = df["email"].str.lower()` `df["name"] = df["name"].str.title()`
Map variants	`df["city"] = df["city"].replace({"NYC": "New York"})`
Remove symbols	`df["name"] = df["name"].str.replace(r"[^\w\s]", "", regex=True)`
Keep digits only	`df["phone"] = df["phone"].str.replace(r"\D", "", regex=True)`
Pad with zeros	`df["zip"] = df["zip"].str.zfill(5)`
Wrong length	`df[df["zip"].str.len() != 5]`
Split in two	`df[["first", "last"]] = (df["name"].str.split(" ", n=1, expand=True))`
Pull out a pattern	`df["zip"] = df["address"].str.extract(r"(\d{5})", expand=False)`

WATCH `.str.replace` is literal unless you add `regex=True`.

TYPES & CONVERSION

$1,200 → number	`s = df["price"].str.replace(r"[$,]", "", regex=True)` `df["price"] = pd.to_numeric(s, errors="coerce")`
See what failed	`n = pd.to_numeric(df["qty"], errors="coerce")` `df[n.isna() & df["qty"].notna()]`
Integers + blanks	`df["qty"] = pd.to_numeric(df["qty"], errors="coerce").astype("Int64")`
12% → 0.12	`df["rate"] = pd.to_numeric(df["rate"].str.rstrip("%")) / 100`
Round to 2 places	`df["price"] = df["price"].round(2)`
Number → text	`df["id"] = df["id"].astype("str")`
Many at once	`df = df.astype({"qty": "Int64", "region": "category"})`
Labels → category	`df["region"] = df["region"].astype("category")`
Yes/No → True/False	`df["active"] = (df["active"].str.lower().map({"yes": True, "no": False}))`

WATCH `errors="ignore"` is gone. Blanks? Use `"Int64"`, not `int`.

DATES

Text to dates
```
df["date"] = pd.to_datetime(df["date"],
    format="%Y-%m-%d", errors="coerce")
```

Day first (31/01)
```
df["shipped"] = pd.to_datetime(
    df["shipped"], dayfirst=True)
```

Mixed formats
```
df["when"] = pd.to_datetime(df["when"],
    format="mixed")
```

Pull date parts
```
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month_name()
df["weekday"] = df["date"].dt.day_name()
```

Days between dates
```
df["days"] = (df["shipped"]
    - df["date"]).dt.days
```

Rows in a range
```
df[df["date"].between(
    "2025-01-01", "2025-03-31")]
```

Totals per month
```
(df.resample("ME", on="date")["qty"]
    .sum())
```

WATCH Bad dates turn into `NaT`. Count them with `.isna().sum()`.

FILTER & OUTLIERS

Two rules at once
```
df[(df["price"] > 0)
   & (df["city"] == "NYC")]
```

Same, as a query
```
df.query("price > 0 and city == 'NYC'")
```

Value is in a list
```
df[df["city"].isin(["Boston", "NYC"])]
```

Keep valid rows
```
df = df[df["price"] >= 0]
```

Text starts with
```
df[df["sku"].str.startswith("A")]
```

Text contains
```
df[df["email"].str.contains("@",
    na=False)]
```

Flag outliers (IQR)
```
q1, q3 = df["price"].quantile(
    [0.25, 0.75])
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df["outlier"] = ~df["price"].between(
    lo, hi)
```

See the 5 largest
```
df.nlargest(5, "price")
```

Cap extreme values
```
df["price"] = df["price"].clip(
    lower=0, upper=5000)
```

WATCH Use `&` `|` `~` (not `and` / `or`) and put each rule in brackets.

COMBINE TABLES

Look up a table
```
df = df.merge(products, on="sku",
    how="left", validate="many_to_one")
```

Different key names
```
df.merge(customers, left_on="email",
    right_on="contact", how="left")
```

Did keys match?
```
m = df.merge(products, on="sku",
    how="left", indicator=True)
m["_merge"].value_counts()
```

Stack tables
```
both = pd.concat([jan, feb],
    ignore_index=True)
```

WATCH More rows after a merge? Repeated keys. `validate=` catches it.

RENAME & REORDER COLUMNS

Rename columns
```
df = df.rename(
    columns={"Cust Name": "customer"})
```

Tidy all headers
```
df.columns = (df.columns.str.strip()
    .str.lower().str.replace(" ", "_"))
```

Drop columns
```
df = df.drop(columns=["notes"])
```

Pick and order
```
df = df[["id", "name", "city", "price"]]
```

Column to the front
```
df.insert(0, "id", df.pop("id"))
```

Sort by several
```
df = df.sort_values(["city", "price"],
    ascending=[True, False])
```

Renumber rows
```
df = df.reset_index(drop=True)
```

RESHAPE & SUMMARISE

Totals per group
```
df.groupby("city", as_index=False).agg(
    orders=("id", "count"),
    units=("qty", "sum"))
```

Pivot table
```
df.pivot_table(index="city",
    columns="region", values="qty",
    aggfunc="sum", fill_value=0)
```

Count combinations
```
pd.crosstab(df["city"], df["region"])
```

Wide to long
```
long = sales.melt(id_vars="city",
    var_name="month",
    value_name="units")
```

Long to wide
```
long.pivot(index="city",
    columns="month", values="units")
```

CHECK & SAVE

Stop if a rule fails
```
assert df["id"].is_unique
assert df["email"].notna().all()
```

Check final types
```
df.dtypes
```

Save the result
```
df.to_csv("clean.csv", index=False)
df.to_excel("clean.xlsx", index=False)
df.to_parquet("clean.parquet")
```

WATCH CSV forgets types (dates come back as text). Parquet keeps them.

PANDAS HABITS

copy-on-write is always on

Assign back, never chain
```
NO  df["price"][df["price"] < 0] = 0
YES df.loc[df["price"] < 0, "price"] = 0
```

Methods return new data
```
NO  df["city"].str.strip()
YES df["city"] = df["city"].str.strip()
```

Skip inplace=True
```
NO  df.dropna(inplace=True)
YES df = df.dropna()
```

Pick text columns as str
```
NO  df.select_dtypes("object")
YES df.select_dtypes("str")
```